

Tarea #1

05.2026.IC.90501.G1.TECNICAS DE
PROGRAMACION

Stephanie Solis

*Entregable #1 Actualizado post
implementación*

Proyecto #1 – Nutrición para todos

Introducción.....	5
Descripción del Proyecto.....	5
Objetivos del Proyecto.....	6
Objetivo Principal.....	6
Objetivos Secundarios.....	6
Objetivos del Sistema.....	7
Arquitectura del Sistema MVC.....	7
Backlog del Proyecto.....	8
EPIC 1 – Gestión de Usuarios.....	8
PBI 1.1.1 – Crear Usuario.....	8
User Story.....	8
Criterios de aceptación.....	9
Subtareas técnicas.....	9
PBI 1.1.2 – Editar Usuario.....	9
User Story.....	9
Criterios de aceptación.....	9
Subtareas técnicas.....	10
PBI 1.1.3 – Eliminar Usuario.....	10
User Story.....	10
Criterios de aceptación.....	10
Subtareas técnicas.....	10
EPIC 2 – Gestión de Productos.....	11
PBI 2.1.1 – Crear Producto.....	11
User Story.....	11
Criterios de aceptación.....	11
Subtareas técnicas.....	11
PBI 2.1.2 – Editar Producto.....	12
User Story.....	12
Criterios de aceptación.....	12
Subtareas técnicas.....	12
PBI 2.1.3 – Eliminar Producto.....	12
User Story.....	12
Criterios de aceptación.....	13
Subtareas técnicas.....	13
EPIC 3 – Gestión de Menús.....	13
PBI 3.1.1 – Crear Menú Diario.....	13
User Story.....	13
Criterios de aceptación.....	13

Subtareas técnicas.....	13
PBI 3.1.2 – Agregar Productos al Menú.....	14
User Story.....	14
Criterios de aceptación.....	14
Subtareas técnicas.....	14
EPIC 4 – Información Nutricional.....	15
PBI 4.1.1 – Calcular IMC.....	15
User Story.....	15
Criterios de aceptación.....	15
Subtareas técnicas.....	15
PBI 4.1.2 – Calcular Calorías y carboos.....	15
User Story.....	15
Criterios de aceptación.....	15
Subtareas técnicas.....	16
EPIC 5 – Estadísticas.....	16
PBI 5.1.1 – Consumo vs Meta.....	16
User Story.....	16
Criterios de aceptación.....	16
Subtareas técnicas.....	16
PBI 5.1.2 – Estadísticas por Rango.....	17
User Story.....	17
Criterios de aceptación.....	17
Subtareas técnicas.....	17
EPIC TÉCNICO – Infraestructura y Calidad.....	17
Objetivo.....	17
Propósito.....	17
Alcance.....	18
TEC 1 – Configurar estructura MVC.....	18
TEC 2 – Configurar proyecto Avalonia UI.....	18
TEC 3 – Configurar persistencia base.....	19
TEC 4 – Manejo global de excepciones.....	19
TEC 5 – Refactorización.....	19
TEC 6 – Pruebas integrales.....	20
Sprints - Planificación y metodología.....	20
Control de cambios e implementación en EPIC y PBIS.....	21
Resumen general.....	21
Cambios por EPIC.....	22
EPIC 1 – Gestión de Usuarios.....	22
EPIC 2 – Gestión de Productos.....	22

EPIC 3 – Gestión de Menús.....	23
EPIC 4 – Información Nutricional.....	23
EPIC 5 – Estadísticas.....	24
EPIC Técnico – Infraestructura y Calidad.....	25
Conclusión del control de cambios.....	26
Riesgos del Proyecto.....	26
Conclusiones / Análisis de Aprendizaje.....	26

Introducción

El presente documento corresponde al Entregable #1 del Proyecto #1 – Nutrición para Todos, desarrollado en el curso Técnicas de Programación. En esta etapa se define la planificación integral del sistema mediante la estructuración del trabajo en Epics y Product Backlog Items (PBIs), estableciendo el alcance funcional y técnico del proyecto.

El sistema, denominado Dragon Nutrex, es una aplicación de escritorio desarrollada en C# utilizando Avalonia UI, cuyo propósito es permitir la gestión de información nutricional de manera organizada y automatizada. La solución contempla la administración de usuarios, productos alimenticios, menús diarios y estadísticas, incorporando cálculos como IMC, calorías objetivo y distribución de macronutrientes.

Aunque el lineamiento inicial indicaba el uso de Windows Forms, se optó por utilizar Avalonia UI debido a su mayor flexibilidad, modernidad y compatibilidad multiplataforma, manteniendo completamente la arquitectura MVC y los principios de diseño establecidos en el proyecto.

Desde el enfoque técnico, el proyecto se basa en la arquitectura Modelo-Vista-Controlador (MVC), complementada con capas de servicios y repositorios, garantizando una adecuada separación de responsabilidades, mantenibilidad, escalabilidad y claridad estructural del sistema. Asimismo, se incorporan buenas prácticas de desarrollo como SOLID y Clean Code.

Descripción del Proyecto

Dragon Nutrex es una aplicación de escritorio desarrollada en C# utilizando Avalonia UI que permite la gestión integral de información nutricional.

El sistema permite:

- Administrar usuarios con información física y objetivos

- Gestionar productos alimenticios con valores nutricionales
- Crear menús diarios con múltiples productos
- Calcular automáticamente calorías y macronutrientes
- Analizar el consumo nutricional en el tiempo

Adicionalmente, se implementó un módulo avanzado de estadísticas que permite evaluar el comportamiento del usuario en un rango de fechas, comparar su consumo con diferentes tipos de dieta y generar recomendaciones automáticas.

Objetivos del Proyecto

Objetivo Principal

Desarrollar una aplicación de escritorio en C# utilizando Windows Forms bajo arquitectura MVC, aplicando principios SOLID, prácticas de Clean Code y buenas prácticas de ingeniería de software, que permita la gestión integral de información nutricional mediante operaciones CRUD y garantice una estructura mantenible y escalable.

Objetivos Secundarios

- Diseñar e implementar la arquitectura MVC integrando principios SOLID, estándares de Clean Code y manejo adecuado de excepciones, asegurando separación de responsabilidades y calidad estructural del sistema.
- Desarrollar funcionalidades CRUD para la administración de usuarios, productos y menús, incorporando validaciones de datos y persistencia estructurada (CSV/JSON).

- Entregar documentación técnica y planificación estructurada del sistema (Epics, PBIs, criterios de aceptación y backlog), evidenciando la correcta aplicación de metodologías ágiles y buenas prácticas de desarrollo.

Objetivos del Sistema

Proveer una solución de software de escritorio que permita gestionar de manera estructurada la información nutricional de los usuarios, mediante la administración de perfiles, productos y menús diarios, incorporando cálculos automáticos y estadísticas comparativas, bajo una arquitectura MVC que garantice calidad, mantenibilidad y correcta aplicación de principios de ingeniería de software.

Arquitectura del Sistema MVC

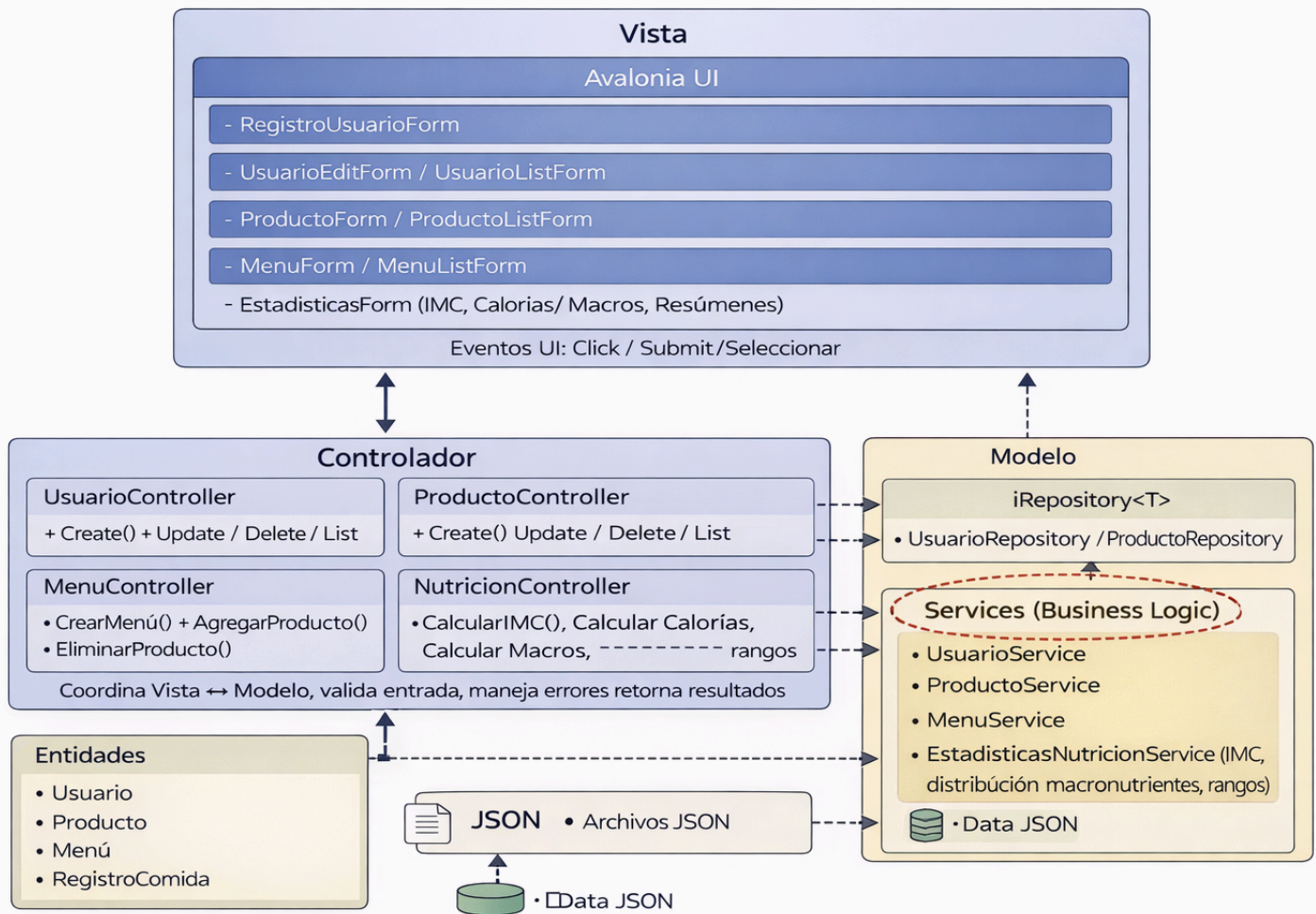
La arquitectura del sistema sigue el patrón MVC extendido:

UI (Avalonia) → Controller → Service → Repository → JSON

- UI (Avalonia): Gestiona la interacción con el usuario (formularios, botones, visualización de datos).
- Controllers: Reciben las acciones de la UI y coordinan las operaciones del sistema.
- Services - Contienen la lógica de negocio: cálculos nutricionales, validaciones, reglas del sistema, repositories, manejan la persistencia de datos.
- JSON (Data): Almacenamiento de información del sistema.

Esta arquitectura garantiza:

- Separación de responsabilidades
- Bajo acoplamiento
- Alta cohesión
- Facilidad de mantenimiento



Backlog del Proyecto

EPIC 1 – Gestión de Usuarios

PBI 1.1.1 – Crear Usuario

User Story

Como usuario del sistema, quiero registrar mis datos personales y nutricionales,

para que el sistema pueda calcular mis metas y realizar seguimiento.

Criterios de aceptación

1. Permite ingresar peso, altura, actividad, objetivo y tipo de dieta.
2. Valida campos obligatorios.
3. No permite valores negativos.
4. Guarda la información en archivo.
5. Recupera los datos al reiniciar la aplicación.

Subtareas técnicas

- Crear clase Usuario (Model)
- Definir propiedades y validaciones básicas
- Crear formulario RegistroUsuarioForm
- Implementar UsuarioController
- Implementar UsuarioService
- Implementar UsuarioRepository
- Implementar método Create/GuardarUsuario()
- Persistencia en archivo
- Manejo de excepciones
- Pruebas manuales

PBI 1.1.2 – Editar Usuario

User Story

Como usuario del sistema, quiero modificar mis datos personales y nutricionales, para mantener mi información actualizada y que los cálculos reflejen cambios recientes.

Criterios de aceptación

1. Permite seleccionar un usuario existente.
2. Permite modificar peso, altura, actividad, objetivo y tipo de dieta.
3. Valida campos obligatorios.

4. No permite valores negativos.
5. Actualiza la información en el archivo.
6. Los cambios se mantienen al reiniciar la aplicación.

Subtareas técnicas

- Implementar método ActualizarUsuario()
- Crear pantalla de edición
- Cargar datos existentes en formulario
- Reutilizar validaciones existentes
- Actualizar persistencia
- Refrescar vista tras guardar
- Manejo de excepciones
- Pruebas manuales

PBI 1.1.3 – Eliminar Usuario

User Story

Como usuario del sistema, quiero eliminar un perfil de usuario, para mantener actualizada la información y evitar registros innecesarios.

Criterios de aceptación

1. Permite seleccionar un usuario existente para eliminar.
2. Solicita confirmación antes de eliminar el usuario.
3. Elimina correctamente el usuario del archivo de persistencia.
4. El usuario eliminado deja de aparecer en el sistema.
5. Maneja adecuadamente la eliminación de datos asociados (menús y registros).

Subtareas técnicas

- Implementar método EliminarUsuario() en UsuarioController
- Implementar método Delete() en UsuarioRepository
- Actualizar persistencia de datos
- Validar eliminación de datos asociados (menús/registros)

- Confirmación en la interfaz de usuario
- Refrescar listado tras eliminación
- Manejo de excepciones
- Pruebas funcionales

EPIC 2 – Gestión de Productos

PBI 2.1.1 – Crear Producto

User Story

Como usuario, quiero registrar productos con valores nutricionales, para utilizarlos en mis menús diarios.

Criterios de aceptación

1. Permite ingresar nombre y carbos.
2. Valida que carbos ≥ 0 .
3. Guarda el producto correctamente.
4. Aparece en el listado dinámico.

Subtareas técnicas

- Crear clase Producto
- Definir propiedades nutricionales
- Implementar validaciones (carbos ≥ 0)
- Crear formulario ProductoForm
- Implementar ProductoController
- Implementar ProductoService
- Implementar ProductoRepository
- Método GuardarProducto()
- Persistencia en archivo

- Pruebas funcionales

PBI 2.1.2 – Editar Producto

User Story

Como usuario, quiero modificar la información de un producto, para corregir o actualizar sus valores nutricionales.

Criterios de aceptación

1. Permite seleccionar un producto existente.
2. Permite modificar nombre y valores nutricionales.
3. Valida que carbo ≥ 0 .
4. Guarda correctamente los cambios.
5. Actualiza el listado dinámico.

Subtareas técnicas

- Implementar método ActualizarProducto()
- Cargar producto en formulario
- Validaciones de datos
- Actualizar persistencia de datos
- Refrescar listado en UI
- Manejo de excepciones
- Pruebas funcionales

PBI 2.1.3 – Eliminar Producto

User Story

Como usuario, quiero eliminar un producto, para mantener actualizado el catálogo de alimentos.

Criterios de aceptación

1. Permite seleccionar un producto para eliminar.
2. Solicita confirmación antes de eliminar.
3. El producto deja de aparecer en el listado.
4. Se actualiza la persistencia de datos.

Subtareas técnicas

- Implementar método EliminarProducto()
- Confirmación en UI
- Actualizar persistencia
- Refrescar listado
- Manejo de excepciones
- Pruebas manuales

EPIC 3 – Gestión de Menús

PBI 3.1.1 – Crear Menú Diario

User Story

Como usuario, quiero crear un menú por fecha, para registrar mis comidas diarias.

Criterios de aceptación

1. Permite seleccionar fecha.
2. Evita duplicados por fecha.
3. Guarda correctamente el menú.

Subtareas técnicas

- Crear clase Menu
- Relacionar Usuario con Menu

- Validar duplicado por fecha
- Crear formulario de selección
- Implementar MenuController
- Implementar persistencia
- Pruebas manuales

PBI 3.1.2 – Agregar Productos al Menú

User Story

Como usuario, quiero agregar productos a un menú diario, para registrar mis comidas y calcular mis totales nutricionales.

Criterios de aceptación

1. Permite seleccionar producto y cantidad.
2. Valida que cantidad > 0.
3. Calcula automáticamente calorías.
4. Calcula automáticamente macronutrientes.
5. Actualiza totales en tiempo real.
6. Guarda el registro correctamente.

Subtareas técnicas

- Crear clase RegistroComida
- Implementar relación Menu-Producto
- Implementar cálculo automático de calorías
- Implementar cálculo automático de macronutrientes
- Validar cantidad > 0
- Actualizar totales en tiempo real
- Persistencia de registros
- Refrescar vista
- Pruebas funcionales

EPIC 4 – Información Nutricional

PBI 4.1.1 – Calcular IMC

User Story

Como usuario, quiero visualizar mi IMC, para conocer un indicador básico de salud.

Criterios de aceptación

1. Calcula IMC correctamente.
2. Maneja división por cero.
3. Muestra resultado en pantalla.

Subtareas técnicas

- Implementar método CalcularIMC()
- Validar altura > 0
- Mostrar resultado en UI
- Pruebas de cálculo

PBI 4.1.2 – Calcular Calorías y carbo

User Story

Como usuario, quiero conocer mis calorías recomendadas y distribución de macronutrientes, para comparar con mi consumo diario.

Criterios de aceptación

1. Calcula calorías recomendadas según objetivo.
2. Calcula distribución de carbo según tipo de dieta.
3. Muestra resultados en pantalla.
4. Se actualiza automáticamente si cambian los datos del usuario.

Subtareas técnicas

- Implementar método Calcular Calorias Objetivo()
- Implementar método CalcularDistribucioncarbos()
- Aplicar reglas según tipo de dieta
- Mostrar resultados en UI
- Actualización automática si cambia perfil
- Pruebas de cálculo

EPIC 5 – Estadísticas

PBI 5.1.1 – Consumo vs Meta

User Story

Como usuario, quiero comparar mi consumo diario con mi meta, para monitorear mi progreso.

Criterios de aceptación

1. Muestra calorías consumidas.
2. Muestra diferencia respecto a la meta.
3. Muestra carbohidratos consumidos.

Subtareas técnicas

- Implementar método ObtenerResumenDiario()
- Calcular diferencia vs meta
- Mostrar resultados en UI
- Manejar caso sin registros
- Pruebas funcionales

PBI 5.1.2 – Estadísticas por Rango

User Story

Como usuario, quiero visualizar estadísticas en un rango de fechas, para analizar mi progreso nutricional.

Criterios de aceptación

1. Permite seleccionar fecha inicio y fecha fin.
2. Valida que fecha inicio $\leq$ fecha fin.
3. Calcula totales en el rango seleccionado.
4. Calcula promedios de consumo.
5. Muestra resultados en pantalla.

Subtareas técnicas

- Implementar método ObtenerResumenPorRango()
- Validar fecha inicio $\leq$ fecha fin
- Calcular totales
- Calcular promedios
- Mostrar resultados en pantalla
- Pruebas funcionales

EPIC TÉCNICO – Infraestructura y Calidad

Objetivo

Establecer la base técnica del sistema Dragon Nutrex asegurando una arquitectura organizada, mantenible y alineada a buenas prácticas de desarrollo.

Propósito

Garantizar que el sistema tenga una estructura sólida que permita escalabilidad, control de errores y correcta separación de responsabilidades.

Alcance

- Configuración de arquitectura MVC
- Implementación de persistencia por módulo
- Manejo global de excepciones
- Aplicación de principios SOLID
- Pruebas integrales del sistema

TEC 1 – Configurar estructura MVC

- Crear solución/proyecto y estructura de carpetas:
Models/Views/Controllers/Services/Repositories/Utils
- Definir convención de nombres (Forms, Controllers, Services, Repositories)
- Crear clases base del dominio vacías: Usuario, Producto, Menu, RegistroComida
- Crear Controllers base (stubs): UsuarioController, ProductoController, MenuController, NutricionController
- Crear Services base (stubs): UsuarioService, ProductoService, MenuService, NutricionService
- Crear Repositories base (stubs): UsuarioRepository, ProductoRepository, MenuRepository
- Validar separación de responsabilidades (no lógica en Forms, lógica en Services)

TEC 2 – Configurar proyecto Avalonia UI

- Crear proyecto Avalonia UI y validar ejecución
- Configurar Program.cs (entry point)
- Crear MainForm (menú principal / navegación)
- Definir navegación básica (botones/menú a: Usuarios, Productos, Menús, Nutrición, Estadísticas)
- Crear Forms placeholders (vacíos) para cada módulo:
 - RegistroUsuarioForm, UsuarioListForm
 - ProductoForm, ProductoListForm
 - MenuForm, MenuListForm
 - NutricionForm (IMC/carbos)

- EstadisticasForm
- Prueba: abrir/cerrar formularios sin errores

TEC 3 – Configurar persistencia base

- Definir interfaz genérica IRepository<T>
(Create/Update/Delete/GetAll/GetById)
- Definir modelo de almacenamiento (JSON o CSV) y estructura de carpetas
/Data
- Implementar clase helper FileStorage (leer/escribir archivo)
- Implementar repositorio base genérico (si aplica) o repos base por entidad
- Implementar persistencia inicial para Usuarios (para validar end-to-end)
- Implementar métodos de serialización/deserialización
- Manejo básico de errores de archivo (archivo no existe, vacío, corrupto)
- Prueba: guardar y cargar datos tras reiniciar app

TEC 4 – Manejo global de excepciones

- Configurar handler global (AppDomain/ThreadException para Avalonia UI)
- Crear clase Logger (guardar logs en archivo)
- Definir formato de log (fecha, módulo, excepción, mensaje)
- Implementar mensajes amigables al usuario (MessageBox)
- Centralizar manejo de validaciones (excepciones controladas vs no controladas)
- Probar escenarios:
 - Archivo no existe
 - Error de parseo JSON/CSV
 - Valores inválidos (negativos, cero, etc.)
- Verificar que la app no se cae ante errores comunes

TEC 5 – Refactorización

- Identificar y eliminar duplicación (validaciones repetidas, lectura/escritura repetida)
- Extraer utilidades comunes (validaciones, conversión, helpers)

- Revisar responsabilidades (Forms sin lógica, funcionamiento de Controllers, Services calculan, Repos guardan)
- Aplicar principios SOLID
- Renombrar clases/métodos para consistencia
- Limpiar warnings / code smells
- Revisión final del flujo MVC (dependencias correctas)

TEC 6 – Pruebas integrales

- Definir checklist de pruebas por flujo completo:
 - Crear/Editar/Eliminar Usuario
 - Crear/Editar/Eliminar Producto
 - Crear Menú por fecha (sin duplicados)
 - Agregar productos al menú (cantidad > 0, cálculos)
 - Calcular IMC / calorías / carbos
 - Estadísticas: consumo vs meta y por rango
- Probar persistencia completa (cerrar app y reabrir)
- Probar casos borde:
 - Sin usuarios / sin productos / sin menús
 - Fechas inválidas (inicio > fin)
 - Cantidad 0 o negativa
- Registrar evidencias (capturas o documento breve de resultados)
- Corrección de bugs encontrados + rerun de pruebas
- Validación final para entrega/demo

Sprints - Planificación y metodología

El proyecto Dragon Nutrex se desarrollará en cuatro sprints organizados de manera incremental, permitiendo construir el sistema por módulos funcionales y asegurar avances progresivos y controlados. Cada sprint tendrá objetivos definidos y entregables específicos alineados con la arquitectura MVC.

La gestión del trabajo se realizará en JIRA bajo la metodología Scrum, utilizando Epics, PBIs y subtarefas para planificar, priorizar y dar seguimiento al desarrollo, garantizando organización, trazabilidad y control del progreso del proyecto.

Sprint	Fechas	Contenido (Epics y PBIs)
Sprint 1	8 - 13 marzo	EPIC Técnico (TEC 1, TEC 2, TEC 3) + EPIC 1 - Gestión de Usuarios (PBI 1.1.1, 1.1.2, 1.1.3)
Sprint 2	14 - 20 marzo	EPIC 2 - Gestión de Productos (PBI 2.1.1, 2.1.2, 2.1.3)
Sprint 3	21 - 27 marzo	EPIC 3 - Gestión de Menús (PBI 3.1.1, 3.1.2) + EPIC 4 - Información Nutricional (PBI 4.1.1, 4.1.2)
Sprint 4	28 - 30 marzo	EPIC 5 - Estadísticas (PBI 5.1.1, 5.1.2) + mejoras avanzadas de análisis nutricional (histórico, macros dinámicos, comparación por dieta) + EPIC Técnico (TEC 4, TEC 5, TEC 6)

Control de cambios e implementación en EPIC y PBIS

Resumen general

Durante el desarrollo del proyecto se realizaron ajustes y mejoras sobre la planificación inicial definida en los EPICs y PBIs, con el objetivo de fortalecer la funcionalidad del sistema, mejorar la experiencia del usuario y asegurar una correcta aplicación de la arquitectura propuesta.

Los cambios implementados no alteran el alcance funcional original, sino que lo amplían y optimizan mediante mejoras técnicas y de usabilidad.

Cambios por EPIC

EPIC 1 – Gestión de Usuarios

Cambios implementados

- Se reemplazó el ingreso manual de Tipo de Dieta (TextBox) por un ComboBox controlado
- Se integró la lista de dietas desde el sistema de estadísticas para mantener consistencia
- Se agregó sincronización entre:
 - Selección de usuario
 - Precarga de datos en UI
- Se reforzaron validaciones de entrada

Impacto

- Mejora la integridad de datos
- Evita errores de escritura
- Asegura coherencia entre módulos

EPIC 2 – Gestión de Productos

Cambios implementados

- Se extendió el modelo de producto para incluir:
 - Calorías
 - Proteínas
 - Carbohidratos
 - Grasas
- Se ajustó la UI para reflejar todos los macronutrientes
- Se mejoraron validaciones numéricas

Impacto

- Permite cálculos nutricionales completos
- Mejora precisión del sistema

EPIC 3 – Gestión de Menús

Cambios implementados

- Se agregó funcionalidad de:
- Edición de cantidades de productos en el menú
- Eliminación de registros individuales
- Eliminación completa de menús
- Se implementó actualización dinámica de totales nutricionales
- Se mejoró la selección y carga de menús existentes
- Se resolvió conflicto de nombres (Menu vs Avalonia.Menu)

Impacto

- Mejora la experiencia de usuario
- Permite edición real del menú sin recrearlo
- Reduce fricción en el uso del sistema

EPIC 4 – Información Nutricional

Cambios implementados

- Se implementó cálculo de IMC con validación segura
- Se mejoró el cálculo de calorías objetivo según perfil
- Se reemplazó cálculo simple de macros por un modelo dinámico:
- Basado en tipo de dieta:
 - Balanceada
 - Vegetariana

- Vegana
- Alta proteína
- Keto
- Ajustado al peso del usuario

Impacto

- Resultados más realistas
- Mayor personalización
- Mejora la calidad del análisis nutricional

EPIC 5 – Estadísticas

Cambios implementados:

- Análisis histórico por usuario
- Selección de rango de fechas
- Cálculo de totales nutricionales
- Cálculo de promedios diarios
- Comparación contra dietas configurables
- Precarga automática de dieta del usuario
- Opción de cambiar dieta para simulación
- Cálculo de diferencias nutricionales
- Generación de recomendaciones automáticas
- Visualización: Se incorporaron indicadores visuales:
 - ● Verde → dentro del rango
 - ● Naranja → desviación moderada
 - ● Rojo → desviación significativa

Impacto

- Convierte el sistema en una herramienta analítica
- Permite simulación nutricional

- Mejora la interpretación de resultados

EPIC Técnico – Infraestructura y Calidad

Cambios implementados

- Se migró la UI de Windows Forms a Avalonia UI
- Se mantuvo la arquitectura MVC extendida
- Se reforzó la separación de capas:
 - UI sin lógica de negocio
 - Services con cálculos
 - Repositories con persistencia
 - Se implementó persistencia completa en JSON
- Se aplicaron principios SOLID:
 - Uso de interfaces (IRepository)
 - Desacoplamiento de servicios
 - Se mejoró el manejo de excepciones
 - Se incorporaron prácticas de Clean Code

Impacto

- Mayor mantenibilidad
- Mejor estructura del proyecto
- Mayor escalabilidad
- Código más limpio y organizado

Conclusión del control de cambios

Los cambios realizados durante la implementación permitieron evolucionar el sistema desde una solución funcional básica hacia una aplicación más robusta, flexible y alineada con buenas prácticas de ingeniería de software.

En particular, el módulo de estadísticas se convirtió en el componente más avanzado del sistema, integrando análisis histórico, cálculos dinámicos y visualización mejorada, lo que aporta un valor significativo al usuario final.

Riesgos del Proyecto

- Errores en validaciones.
- Problemas en persistencia de datos.
- Tiempo limitado para pruebas.

Conclusiones / Análisis de Aprendizaje

El desarrollo del proyecto permitió aplicar de forma práctica la arquitectura por capas basada en MVC, logrando una clara separación de responsabilidades entre interfaz, lógica de negocio y persistencia.

Se implementó un sistema completo que incluye gestión de usuarios, productos y menús, así como un módulo avanzado de estadísticas nutricionales que permite analizar el comportamiento del usuario en el tiempo y compararlo contra diferentes dietas.

El uso de principios SOLID permitió desarrollar una solución mantenible y escalable, mientras que la aplicación de Clean Code facilitó la organización y claridad del código.

Asimismo, el uso de herramientas como Git y Jira permitió llevar un control adecuado del desarrollo mediante sprints, mejorando la planificación y trazabilidad del proyecto.

En conjunto, el proyecto demuestra la importancia de una buena arquitectura, planificación estructurada y aplicación de buenas prácticas para lograr soluciones de software robustas y de calidad.